

Advanced Machine Learning

DATA 642

Lecture 13 — Faster Optimizers

*Figures and notes are from the book Mathematics for Machine Learning by A. Aldo Faisal, Cheng Soon Ong, and Marc Peter Deisenroth, Machine Learning A Bayesian and Optimization Perspective, Sergios Theodoridis, Elsevier, and Hands-On Machine Learning with Scikit-Learn Keras and TensorFlow, Aurelien Geron, O'Reilly

Faster Optimizers

- Training a very large deep neural network can be painfully slow.
- A huge speed boost comes from using a faster optimizer than the regular Gradient Descent optimizer.

Momentum Optimization

- Imagine a bowling ball rolling down a gentle slope on a smooth surface: it will start out slowly, but it will quickly pick up momentum until it eventually reaches terminal velocity. This is the very simple idea behind momentum optimization, proposed by Boris Polyak in 1964.
- Recall that Gradient Descent simply updates the weights by ignoring what the earlier gradients were. If the local gradient is tiny, it goes very slowly.
- Momentum optimization cares a great deal about what previous gradients were:
 - at each iteration, it subtracts the local gradient from the momentum vector $\mathbf{m}$ (multiplied by the learning rate α);
 - it updates the weights by simply adding this momentum vector.

$$\mathbf{m}^{(k+1)} = \gamma \mathbf{m}^{(k)} - \alpha^{(k)} \nabla J \left(\boldsymbol{\theta}^{(k)} \right) \quad (1)$$

and

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + \mathbf{m}^{(k+1)} \quad (2)$$

- Note that if $\gamma = 0.9$, then the terminal velocity is equal to 10 times the gradient times the learning rate, so Momentum optimization ends up going 10 times faster than Gradient Descent! This allows Momentum optimization to escape from plateaus much faster than Gradient Descent.
- In deep neural networks that do not use Batch Normalization, the upper layers will often end up having inputs with very different scales, so using Momentum optimization helps a lot. It can also help roll past local optima.
- Due to the momentum, the optimizer may overshoot a bit, then come back, overshoot again, and oscillate like this many times before stabilizing at the minimum. This is one of the reasons why it is good to have a bit of friction in the system: it gets rid of these oscillations and thus speeds up convergence.
- The one drawback of Momentum optimization is that it adds yet another hyper-parameter to tune. However, the momentum value of 0.9 usually works well in practice and almost always goes faster than regular Gradient Descent.

Nesterov Accelerated Gradient

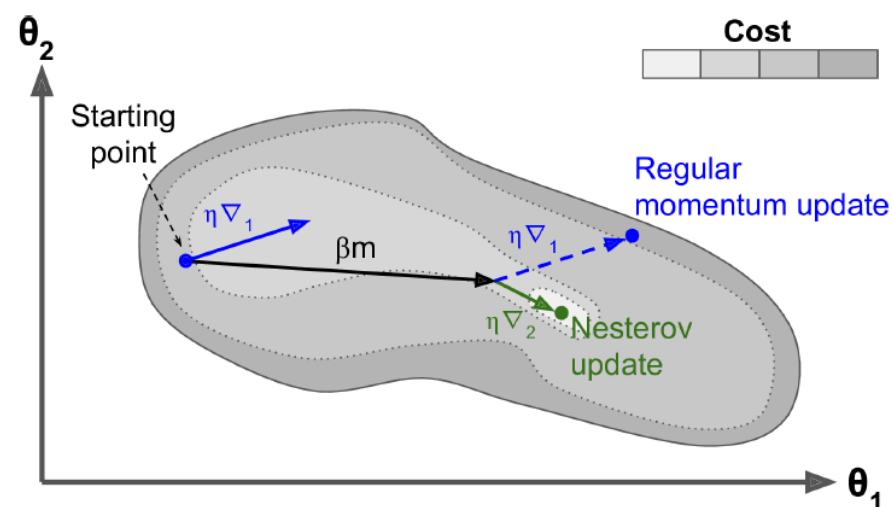
- The idea of Nesterov Momentum optimization, or Nesterov Accelerated Gradient (NAG), is to measure the gradient of the cost function not at the local position but slightly ahead in the direction of the momentum.

$$\mathbf{m}^{(k+1)} = \gamma \mathbf{m}^{(k)} - \alpha^{(k)} \nabla J \left(\boldsymbol{\theta}^{(k)} + \gamma \mathbf{m}^{(k)} \right) \quad (3)$$

and

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + \mathbf{m}^{(k+1)} \quad (4)$$

- This small tweak works because in general the momentum vector will be pointing in the right direction (i.e., toward the optimum), so it will be slightly more accurate to use the gradient measured a bit farther in that direction rather than using the gradient at the original position.



AdaGrad

- Consider the elongated bowl problem again: Gradient Descent starts by quickly going down the steepest slope, then slowly goes down the bottom of the valley. It would be nice if the algorithm could detect this early on and correct its direction to point a bit more toward the global optimum.
- The AdaGrad algorithm achieves this by scaling down the gradient vector along the steepest dimensions

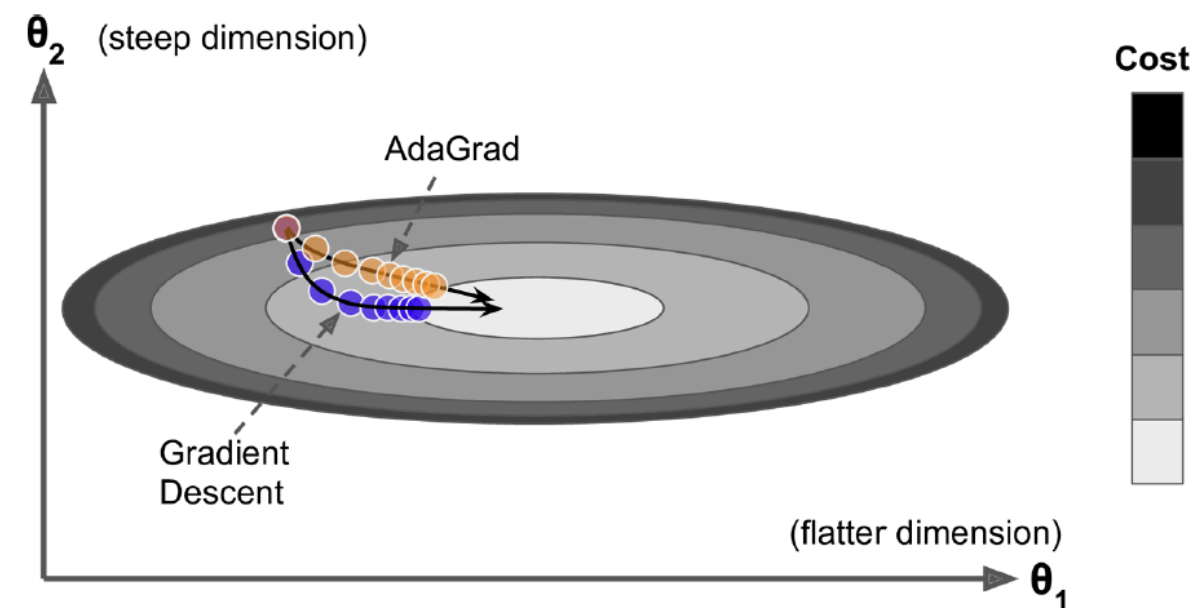
$$\mathbf{s}^{(k+1)} = \mathbf{s}^{(k)} + \nabla J \left(\boldsymbol{\theta}^{(k)} \right) \otimes \nabla J \left(\boldsymbol{\theta}^{(k)} \right) \quad (5)$$

and

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \alpha^{(k)} \nabla J \left(\boldsymbol{\theta}^{(k)} \right) \oslash \sqrt{\mathbf{s}^{(k+1)} + \epsilon} \quad (6)$$

- The first step accumulates the square of the gradients into the vector $\mathbf{s}$ (recall that the $\otimes$ symbol represents the element-wise multiplication).
- The second step is almost identical to Gradient Descent, but with one big difference: the gradient vector is scaled down by a factor of $\sqrt{\mathbf{s}^{(k+1)}} + \epsilon$ (the $\oslash$ symbol represents the element-wise division and ϵ is a smoothing term to avoid division by zero, typically set to 10^{-10}).
- This algorithm decays the learning rate, but it does so faster for steep dimensions than for dimensions with gentler slopes. This is called an adaptive learning rate. It helps point the resulting updates more directly toward the global optimum. One additional benefit is that it requires much less tuning of the learning rate hyper-parameter.

- AdaGrad often performs well for simple quadratic problems, but unfortunately it often stops too early when training neural networks. The learning rate gets scaled down so much that the algorithm ends up stopping entirely before reaching the global optimum. So even though Keras has an Adagrad optimizer, you should not use it to train deep neural networks (it may be efficient for simpler tasks such as Linear Regression, though). However, understanding Adagrad is helpful to grasp the other adaptive learning rate optimizers.



RMSProp

- Although AdaGrad slows down a bit too fast and ends up never converging to the global optimum, the RMSProp algorithm¹⁵ fixes this by accumulating only the gradients from the most recent iterations (as opposed to all the gradients since the beginning of training). It does so by using exponential decay in the first step

$$\mathbf{s}^{(k+1)} = \gamma \mathbf{s}^{(k)} + (1 - \gamma) \nabla J \left(\boldsymbol{\theta}^{(k)} \right) \otimes \nabla J \left(\boldsymbol{\theta}^{(k)} \right) \quad (7)$$

and

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \alpha^{(k)} \nabla J \left(\boldsymbol{\theta}^{(k)} \right) \oslash \sqrt{\mathbf{s}^{(k+1)} + \epsilon} \quad (8)$$

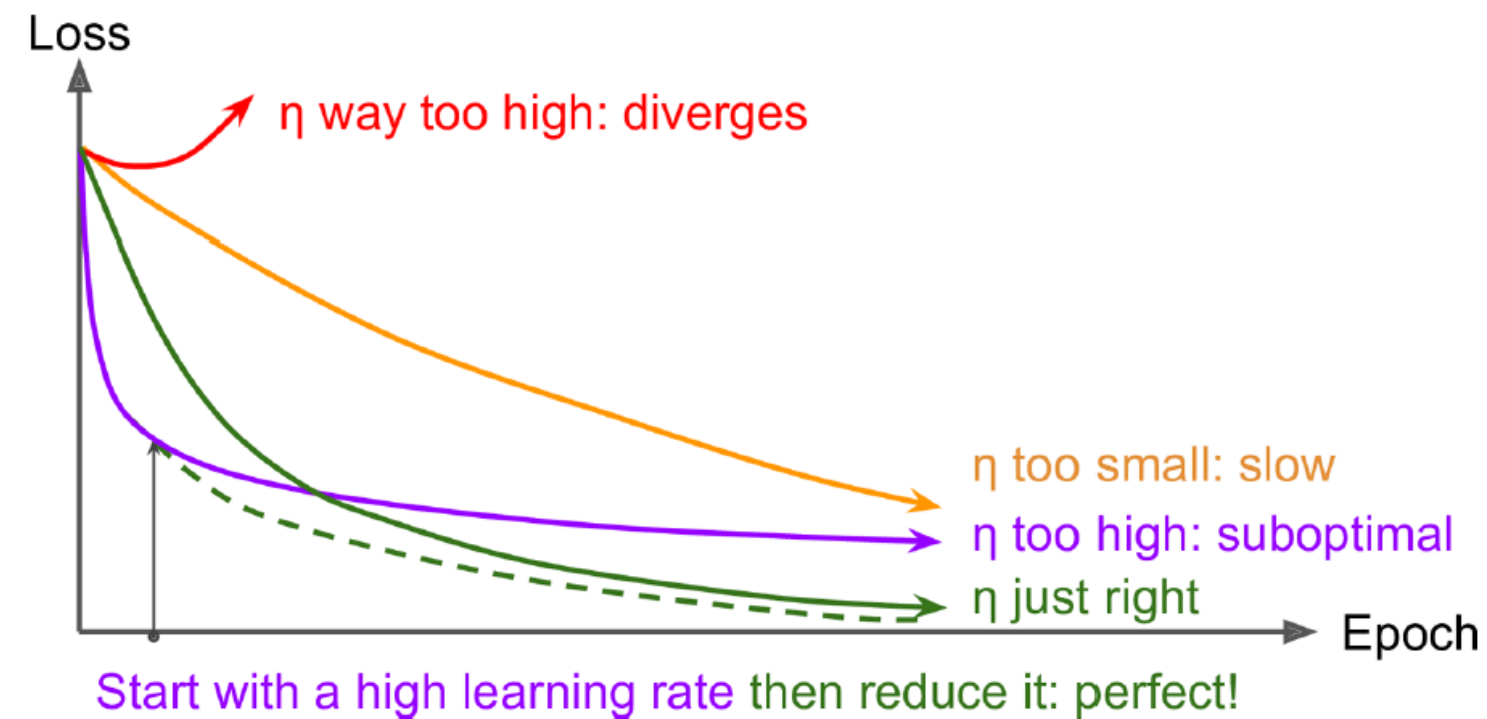
- The decay rate γ is typically set to 0.9.

Important note

- All the optimization techniques discussed so far only rely on the first-order partial derivatives (Jacobians). The optimization literature contains amazing algorithms based on the second-order partial derivatives (the Hessians, which are the partial derivatives of the Jacobians).
- Unfortunately, these algorithms are very hard to apply to deep neural networks because there are n^2 Hessians per output (where n is the number of parameters), as opposed to just n Jacobians per output. Since DNNs typically have tens of thousands of parameters, the second-order optimization algorithms often do not even fit in memory, and even when they do, computing the Hessians is just too slow.

Learning Rate Scheduling

- Finding a good learning rate can be tricky. If learning rate is too low, training will eventually converge to the optimum, but it will take a very long time.



- One approach is to start with a large learning rate, and divide it by 3 until the training algorithm stops diverging. You will not be too far from the optimal learning rate, which will learn quickly and converge to good solution.

There are several alternative approaches that one can use.

- **Power scheduling:** Set the learning rate to a function of the iteration number k . For instance $\alpha^{(k)} = \frac{\alpha^{(0)}}{(1+k/s)^c}$, where s are the steps.
- **Exponential scheduling:** Set the learning rate to $\alpha^{(k)} = \alpha^{(0)} 0.1^{k/s}$
- **Piecewise constant scheduling:** Use a constant learning rate for a number of epochs. For instance, $\alpha^{(0)} = 0.1$ for five epochs, then $\alpha^{(1)} = 0.001$ for 50 epochs and so on.
- **Performance scheduling:** Measure the validation error every N steps (just like for early stopping) and reduce the learning rate by a factor of λ when the error stops dropping.